

Lab 5 — Data Structures

Muhammad Awais

25I-3207

Department of SE-A

Task 1:

Source Code:

```
#include<iostream>
using namespace std;
class Node{
public:
    char val;
    Node *next;
    Node(char val){
        this->val=val;
        next=NULL;
    }
};
class Stack{
    Node *top;
    int limit;
    int length;
public:
    Stack(int limit){
        this->limit=limit;
        top=NULL;
        length=0;
    }
    ~Stack() {
        while (!isEmpty()) {pop();}
    }
    bool isEmpty(){
        if(top==NULL)
        {
            return true;
            cout<<"Stack is empty"<<endl;
        }
        else{return false;}
    }
    bool isFull(){
        if(length==limit)
        {
            cout<<"Stack is full"<<endl;
            return true;
        }
        else{return false;}
    }
};
```

```

    }
    void push(char val){
        if(!isFull())
        {
            Node *temp=new Node(val);
            temp->next=top;
            top=temp;
            length++;
//            cout<<"Element: "<<val<<" is pushed"<<endl;

        }
    }
    void pop(){
        Node *temp=top;
        char val=top->val;
        top=top->next;
        delete temp;
        temp=NULL;
        length--;
//        cout<<"Element: "<<val<<" is popped"<<endl;
    }
    void peek(){if(!isEmpty()){cout<<"Top of the stack: "<<top->val<<endl;}}
    char getTop(){return top->val;}
    bool isMatch(char open, char close){
        if(open=='(' && close==')') return true;
        if(open=='[' && close==']') return true;
        if(open=='{' && close=='}') return true;
        return false;
    }
    void display(){
        if(!isEmpty())
        {
            cout<<"Stack Elements: "<<endl;
            Node *temp=top;
            while(temp!=NULL)
            {
                cout<<temp->val;
                temp=temp->next;
            }
            cout<<endl;
        }
    }
};
int main(){
    string expr;

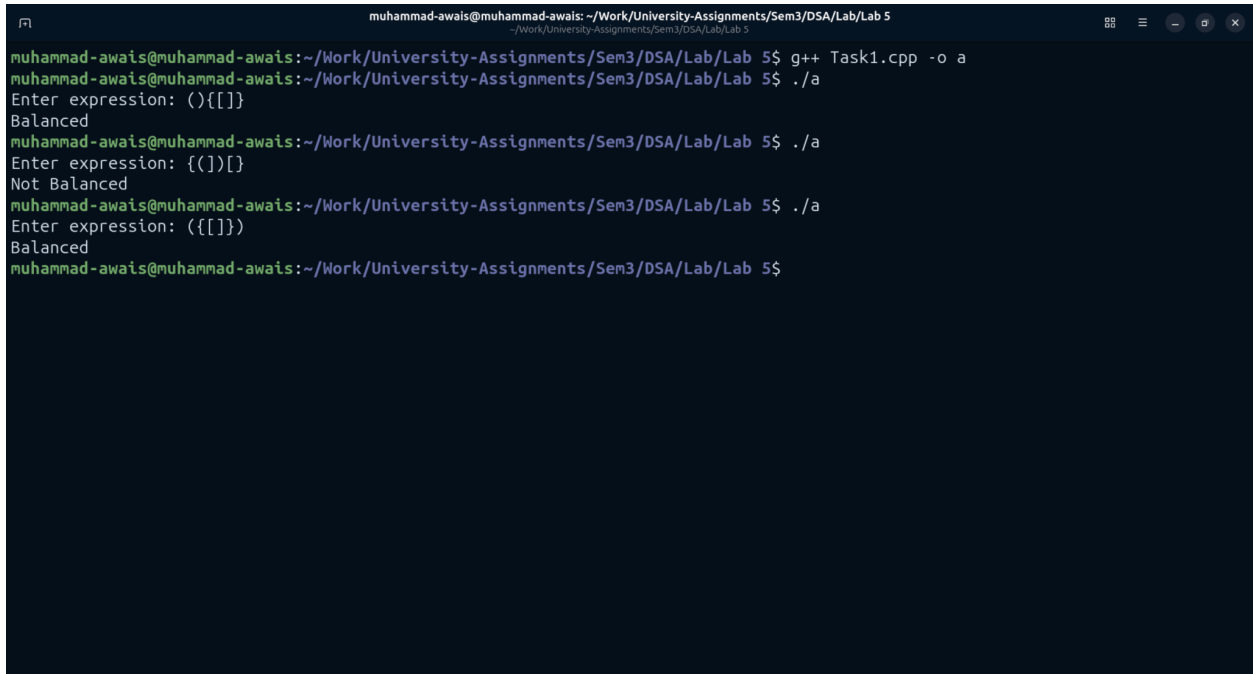
```

```

cout << "Enter expression: ";
getline(cin, expr);
Stack s(expr.length());
bool valid = true;
for(int i = 0; i < expr.length(); i++)
{
    char val = expr[i];
    if(val=='('||val=='['||val=='{')
    {
        s.push(val);
    }
    else if(val==')'||val==']'||val=='}')
    {
        if(s.isEmpty()){valid=false;continue;}
        if(!s.isMatch(s.getTop(),val)){valid=false;continue;}
        s.pop();
    }
    else continue;
}
if(valid && s.isEmpty())cout<<"Balanced"<<endl;
else cout<<"Not Balanced"<<endl;
return 0;
}

```

Screenshot:



```
muhammad-awais@muhammad-awais: ~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ g++ Task1.cpp -o a
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ ./a
Enter expression: (){[]}
Balanced
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ ./a
Enter expression: {(})[]
Not Balanced
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ ./a
Enter expression: ({[]})
Balanced
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$
```

Task 2:

Source Code:

```
#include<iostream>
using namespace std;
class Node{
public:
    int val;
    Node *next;
    Node(int val){
        this->val=val;
        next=NULL;
    }
};
class Stack{
    Node *top;
    int limit;
```

```

        int length;
public:
    Stack(int limit){
        this->limit=limit;
        top=NULL;
        length=0;
    }
    ~Stack() {
        while (!isEmpty()) {pop();}
    }
    bool isEmpty(){
        if(top==NULL){return true;}
        else{return false;}
    }
    bool isFull(){
        if(length==limit)
        {
            cout<<"Stack is full"<<endl;
            return true;
        }
        else{return false;}
    }
    void push(int val){
        if(!isFull())
        {
            Node *temp=new Node(val);
            temp->next=top;
            top=temp;
            length++;
        }
    }
    int pop(){
        Node *temp=top;
        int val=top->val;
        top=top->next;
        delete temp;
        temp=NULL;
        length--;
        return val;
    }
    int getTop(){
        return top->val;
    }
};

```

```

bool isOperator(char val){
    if(val=='+'||val=='-'||val=='*'||val=='/'||val=='%'){return true;}
    return false;
}

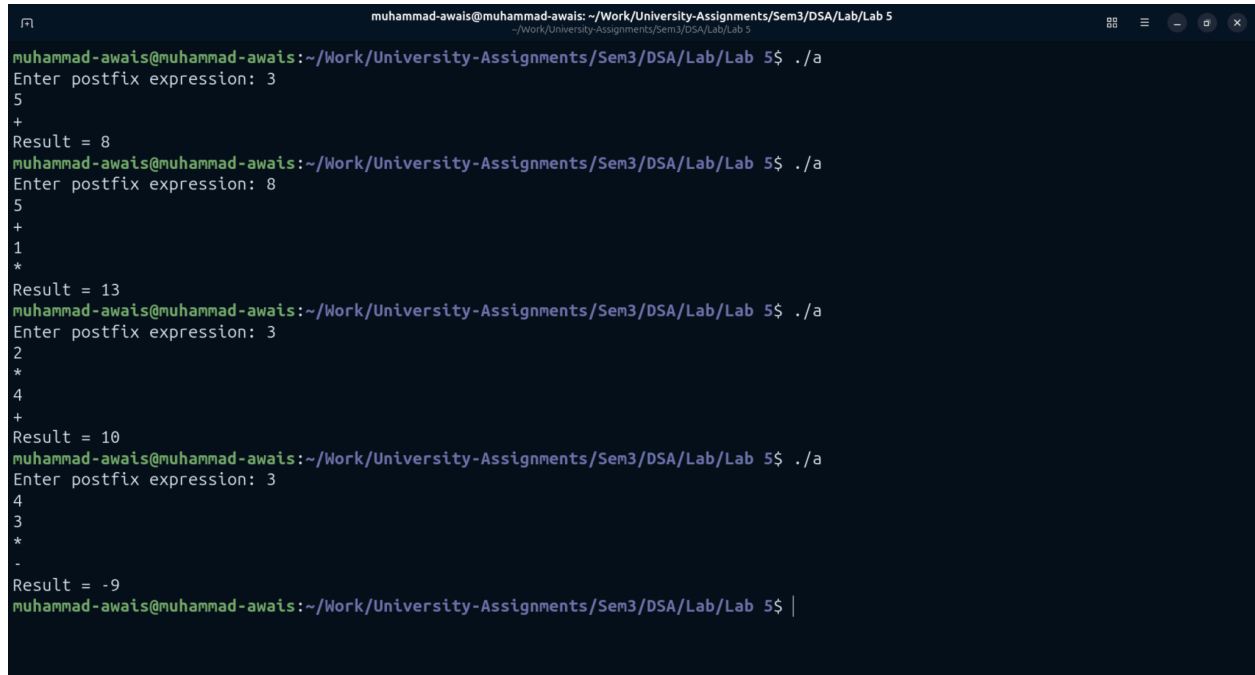
int calculate(int oper1, int oper2, char op){
    if(op=='+')return oper1+oper2;
    if(op=='-')return oper1-oper2;
    if(op=='*')return oper1*oper2;
    if(op=='/')return oper1/oper2;
    return oper1%oper2;
}

int convertInt(string tok){
    int result=0;
    for(int i=0; i<tok.size(); i++)
    {
        result = result*10 + (tok[i]-'0');
    }
    return result;
}

int main(){
    Stack s(15);
    cout << "Enter postfix expression: ";
    string input;
    int oper2,oper1;
    while(cin >> input)
    {
        if(input.size()==1 && isOperator(input[0]))
        {
            oper2=s.pop();
            oper1=s.pop();
            s.push(calculate(oper1,oper2,input[0]));
        }
        else
        {
            s.push(convertInt(input));
        }
    }
    cout << "Result = " << s.getTop() << endl;
    return 0;
}

```

Screenshot:

A screenshot of a terminal window with a dark background. The window title is 'muhammad-awais@muhammad-awais: ~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5'. The terminal shows three separate runs of a postfix calculator program. Each run starts with a prompt 'muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5\$./a', followed by 'Enter postfix expression:'. The first run takes '3' and '5' as input and outputs 'Result = 8'. The second run takes '5', '1', and '*' as input and outputs 'Result = 13'. The third run takes '2', '4', '+', '3', and '-' as input and outputs 'Result = -9'.

```
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ ./a
Enter postfix expression: 3
5
+
Result = 8
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ ./a
Enter postfix expression: 8
5
+
1
*
Result = 13
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ ./a
Enter postfix expression: 3
2
4
+
3
-
Result = -9
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ |
```

Task 3:

Source Code:

```
#include<iostream>
using namespace std;
class Node{
public:
    int val;
    Node *next;
    Node(int val){
        this->val=val;
        next=NULL;
    }
};
class Stack{
    Node *top;
    int limit;
    int length;
public:
```

```

Stack(int limit){
    this->limit=limit;
    top=NULL;
    length=0;
}
~Stack() {
    while (!isEmpty()) {pop();}
}
bool isEmpty(){
    if(top==NULL){return true;}
    else{return false;}
}
bool isFull(){
    if(length==limit)
    {
        cout<<"Stack is full"<<endl;
        return true;
    }
    else{return false;}
}
void push(int val){
    if(!isFull())
    {
        Node *temp=new Node(val);
        temp->next=top;
        top=temp;
        length++;
    }
}
int pop(){
    if(isEmpty()){cout<<"Stack underflow!"<<endl; return 0;}
    Node *temp=top;
    int val=top->val;
    top=top->next;
    delete temp;
    temp=NULL;
    length--;
    return val;
}
int getTop(){
    return top->val;
}
};
bool isOperator(char val){

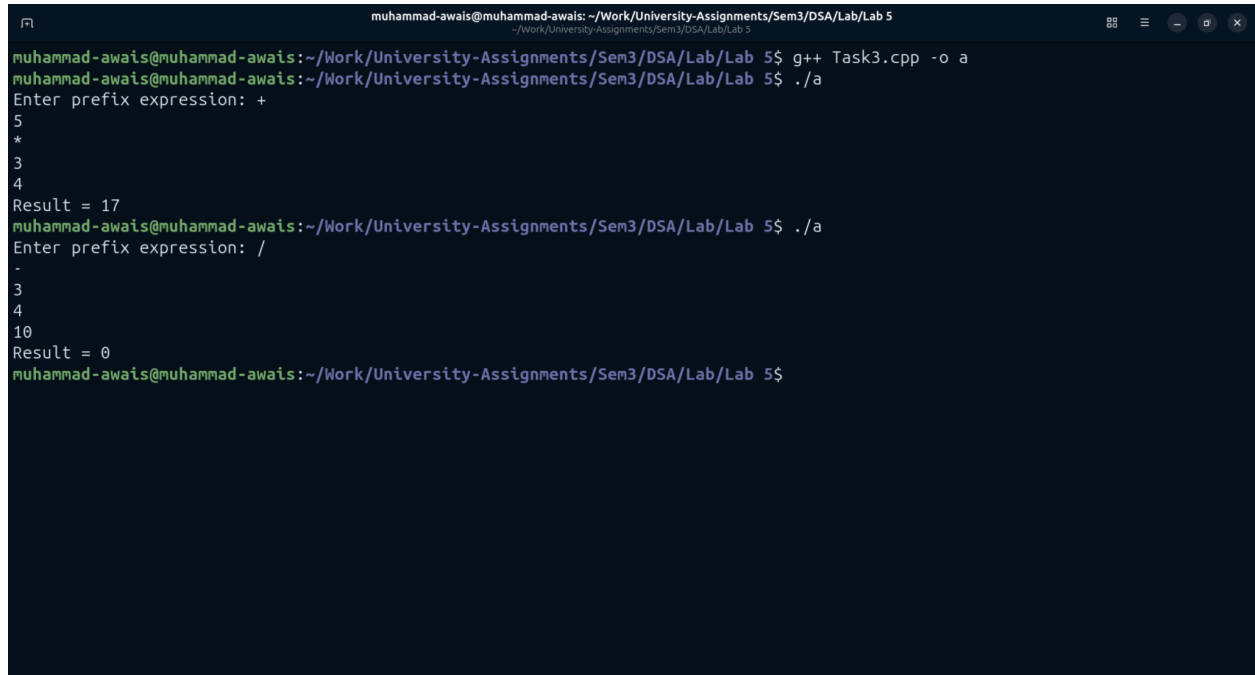
```

```

        if(val=='+'||val=='-'||val=='*'||val=='/'||val=='%'){return true;}
        return false;
    }
    int calculate(int oper1, int oper2, char op){
        if(op=='+')return oper1+oper2;
        if(op=='-')return oper1-oper2;
        if(op=='*')return oper1*oper2;
        if(op=='/')return oper1/oper2;
        return oper1%oper2;
    }
    int convertInt(string tok){
        int result=0;
        for(int i=0; i<tok.size(); i++)
        {
            result = result*10 + (tok[i]-'0');
        }
        return result;
    }
    int main(){
        Stack s(15);
        cout << "Enter prefix expression: ";
        string tokens[100];
        int count=0;
        while(cin >> tokens[count]){count++;}
        int oper2,oper1;
        for(int i=count-1; i>=0; i--)
        {
            string input=tokens[i];
            if(input.size()==1 && isOperator(input[0]))
            {
                oper1=s.pop();
                oper2=s.pop();
                s.push(calculate(oper1,oper2,input[0]));
            }
            else
            {
                s.push(convertInt(input));
            }
        }
        cout << "Result = " << s.getTop() << endl;
        return 0;
    }

```

Screenshot:



```
muhammad-awais@muhammad-awais: ~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ g++ Task3.cpp -o a
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ ./a
Enter prefix expression: +
5
*
3
4
Result = 17
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$ ./a
Enter prefix expression: /
-
3
4
10
Result = 0
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5$
```

The screenshot shows a terminal window with a dark background. The title bar at the top reads "muhammad-awais@muhammad-awais: ~/Work/University-Assignments/Sem3/DSA/Lab/Lab 5". The terminal content shows the compilation of "Task3.cpp" into an executable "a" using "g++". It then shows two runs of the program. The first run takes the postfix expression "+ 5 * 3 4" as input and outputs "Result = 17". The second run takes the postfix expression "/ - 3 4 10" as input and outputs "Result = 0".